

# IncidentIQ Agent Design Report

## Purpose

This report defines the agent architecture that IncidentIQ should build for the hackathon based on:

- the final uploaded architecture using Gradio + FastAPI + LangGraph + n8n + RAG
- the existing notebook prototypes in `notebooks/`
- the planned production structure in `src/`

The goal is not to build the maximum number of agents. The goal is to build the minimum set of agents that produces a convincing multi-agent demo, clear reasoning boundaries, reliable fallback behavior, and a clean implementation story for judges.

## Executive Summary

IncidentIQ should build **eight core agents** and treat the rest of the system as deterministic infrastructure around them.

The best core agent set is:

1. `Classifier Agent`
2. `Timeline Agent`
3. `RCA Agent`
4. `Remediation Agent`
5. `Critic Agent`
6. `Notification Agent`
7. `Ticket Agent`
8. `Cookbook Agent`

Two additional "agent-like" components should exist, but they should be implemented as deterministic services rather than full LLM agents:

1. `Parser / Signal Extraction Layer`
2. `RAG Retrieval Layer`

This split is important for hackathon success. Parsing, evidence extraction, clustering, routing, and reporting should remain deterministic because:

- they are easier to debug
- they run without API keys
- they reduce hallucination risk

- they improve code quality and explainability

LLM agents should only handle ambiguous reasoning, summarization, prioritization, and action generation.

## Recommended Agent System

### 1. Classifier Agent

#### **\*\*Purpose\*\***

Converts clustered evidence into a final incident category and severity.

#### **\*\*Why it should be an agent\*\***

Category and severity often depend on ambiguous evidence and cross-signal interpretation. This is a good use of LLM reasoning, especially when multiple signals overlap.

#### **\*\*Inputs\*\***

- `EvidenceCluster`
- extracted signals
- service name
- top evidence lines
- optional RAG hints

#### **\*\*Outputs\*\***

- incident category
- severity
- confidence
- reasoning summary
- supporting evidence references
- missing evidence notes

#### **\*\*Fallback behavior\*\***

Use deterministic heuristics:

- signal count
- severity keywords
- affected service type
- repeated failure patterns

#### **\*\*Implementation location\*\***

- `src/agents/classifier\_agent.py`

## 2. Timeline Agent

### **\*\*Purpose\*\***

Builds a readable chronology of what happened before, during, and after the incident.

### **\*\*Why it should be an agent\*\***

A plain sorted log list is not enough. The system needs to identify the most important transitions and compress noise into a human-usable sequence.

### **\*\*Inputs\*\***

- cluster evidence
- timestamps
- service names
- trace IDs if present

### **\*\*Outputs\*\***

- ordered timeline events
- short narrative summary
- major turning points

### **\*\*Fallback behavior\*\***

Sort by timestamp and keep representative log lines per stage.

### **\*\*Implementation location\*\***

- `src/agents/timeline\_agent.py`

## 3. RCA Agent

### **\*\*Purpose\*\***

Generates an evidence-backed root cause analysis that separates symptoms from likely cause.

### **\*\*Why it should be an agent\*\***

Root cause reasoning is the strongest "AI moment" in the demo. It is where the project feels materially smarter than rule-only alerting.

### **\*\*Inputs\*\***

- classified incident

- timeline
- evidence cluster
- retrieved SOP/runbook context

#### **\*\*Outputs\*\***

- probable root cause
- alternate hypotheses
- supporting evidence
- missing evidence
- confidence

#### **\*\*Fallback behavior\*\***

Template-based RCA using strongest signal, earliest correlated failure, and category-specific reasoning rules.

#### **\*\*Implementation location\*\***

- `src/agents/rca\_agent.py`

## **4. Remediation Agent**

#### **\*\*Purpose\*\***

Turns the incident analysis into immediate next steps, validation checks, and rollback guidance.

#### **\*\*Why it should be an agent\*\***

This is the action-oriented centerpiece of the system. It also integrates naturally with RAG.

#### **\*\*Inputs\*\***

- incident category and severity
- RCA result
- retrieved SOPs
- system evidence

#### **\*\*Outputs\*\***

- immediate actions
- validation checks
- rollback / safety notes
- estimated blast radius
- required human approvals

#### **\*\*Fallback behavior\*\***

Map category -> known safe actions using deterministic runbook templates.

**\*\*Implementation location\*\***

- `src/agents/remediation\_agent.py`

## 5. Critic Agent

**\*\*Purpose\*\***

Reviews upstream agent outputs for hallucination risk, unsupported claims, and unsafe actions.

**\*\*Why it should be an agent\*\***

This shows mature multi-agent architecture. Judges usually respond well when a system demonstrates self-checking and safety gates.

**\*\*Inputs\*\***

- classifier result
- timeline result
- RCA result
- remediation result
- evidence references

**\*\*Outputs\*\***

- validation findings
- unsupported claims
- missing evidence flags
- unsafe recommendation flags
- pass / revise decision

**\*\*Fallback behavior\*\***

Deterministic checks:

- every claim must cite evidence
- every action must match category/runbook
- every critical incident must include validation steps

**\*\*Implementation location\*\***

- `src/agents/critic\_agent.py`

## 6. Notification Agent

### **\*\*Purpose\*\***

Creates Slack-ready incident communication payloads.

### **\*\*Why it should be an agent\*\***

Stakeholder messaging needs summarization and tone control, not just raw technical output.

### **\*\*Inputs\*\***

- incident summary
- severity
- service
- RCA summary
- remediation summary

### **\*\*Outputs\*\***

- Slack Block Kit JSON
- plain text summary
- stakeholder-safe language

### **\*\*Fallback behavior\*\***

Markdown / JSON template with filled fields.

### **\*\*Implementation location\*\***

- `src/agents/notification\_agent.py`

## **7. Ticket Agent**

### **\*\*Purpose\*\***

Builds JIRA-ready incident tickets for critical issues.

### **\*\*Why it should be an agent\*\***

The structure is partly deterministic, but good ticket descriptions require concise summarization and issue framing.

### **\*\*Inputs\*\***

- incident analysis
- RCA
- remediation
- evidence references

### **\*\*Outputs\*\***

- JIRA issue summary
- JIRA description
- severity labels
- components / service tags

### **\*\*Fallback behavior\*\***

Populate a ticket template using structured fields.

### **\*\*Implementation location\*\***

- `src/agents/ticket\_agent.py`

## **8. Cookbook Agent**

### **\*\*Purpose\*\***

Synthesizes a reusable operational checklist from the incident.

### **\*\*Why it should be an agent\*\***

This is high-value for the hackathon story. It proves the system is not only reactive but also learns operational playbooks from incidents.

### **\*\*Inputs\*\***

- incident summary
- remediation plan
- timeline
- RCA

### **\*\*Outputs\*\***

- triage checklist
- remediation checklist
- validation checklist
- prevention notes

### **\*\*Fallback behavior\*\***

Checklist templates by category and severity.

### **\*\*Implementation location\*\***

- `src/agents/cookbook\_agent.py`

# What Should Not Be Full Agents

## Parser / Log Reader

Do not make this a pure LLM agent in the implementation.

Keep this deterministic:

- file loading
- line parsing
- JSONL parsing
- timestamp extraction
- log level normalization
- regex field extraction

Reason:

- cheaper
- more reliable
- easier to test
- better for malformed logs

Use the UI/demo wording "Log Reader / Classifier Agent" if needed, but in code the parser should stay in ``src/ingest/`` and ``src/signals/``.

## Signal Extractor

Keep deterministic.

This should be rules and heuristics, not free-form reasoning.

## Clusterer

Keep deterministic.

This is state organization, not natural language reasoning.

## RAG Retriever

Treat as a retrieval service, not a conversational agent.

The retriever should return relevant context chunks and the remediation / RCA agents should reason over them.

# Best Multi-Agent Flow



The best production-minded flow for the hackathon is:

1. `Ingest logs`
2. `Normalize events`
3. `Extract signals`
4. `Cluster evidence`
5. `Classifier Agent`
6. `Timeline Agent`
7. `RAG Retrieval`
8. `RCA Agent`
9. `Remediation Agent`
10. `Critic Agent`
11. `Notification Agent`
12. `Ticket Agent`
13. `Cookbook Agent`
14. `Report Builder`

## Conditional Routing

- If `severity in [P1, P2]`: run notification + ticket + cookbook
- If `severity == P3`: run notification preview + cookbook, skip ticket by default
- If `severity == Unknown`: run critic with missing-evidence emphasis and suppress ticket creation
- If LLM fails: use deterministic fallback outputs

## Agent Contracts

Every agent should follow the same pattern:

### Input contract

- strongly typed Pydantic model
- no raw dict-only interfaces across modules
- includes evidence references

### Output contract

- structured model
- machine-readable fields first
- short human summary second
- evidence references always included where relevant

## Runtime rules

- agents never read files directly
- agents never touch ``ground_truth_eval_only``
- agents operate only on typed state passed by the graph
- agents should be pure or near-pure functions from input state to output state

## Detailed Implementation Steps

### Phase 1: Build deterministic foundation

1. Create all Pydantic models under ``src/models/``.
2. Extract ingestion logic from notebook v4 into ``src/ingest/``.
3. Extract rules and signal logic into ``src/signals/``.
4. Build evidence clustering into ``src/clustering/``.
5. Add deterministic report export into ``src/reporting/``.

**\*\*Outcome\*\***

The system can parse logs, detect incident signals, cluster evidence, and emit JSON without any LLM.

### Phase 2: Add core reasoning agents

1. Implement ``ClassifierAgent``.
2. Implement ``TimelineAgent``.
3. Implement ``RCAAgent``.
4. Implement ``RemediationAgent``.

**\*\*Outcome\*\***

The system now produces incident analysis, root cause, and actions.

### Phase 3: Add quality and action agents

1. Implement ``CriticAgent``.
2. Implement ``NotificationAgent``.
3. Implement ``TicketAgent``.
4. Implement ``CookbookAgent``.

**\*\*Outcome\*\***

The system now produces safe, polished, cross-tool outputs.

### Phase 4: Add LangGraph orchestration

1. Create graph state in `src/graph/state.py`.
2. Create node functions in `src/graph/nodes.py`.
3. Create routing rules in `src/graph/edges.py`.
4. Build execution runner in `src/graph/runner.py`.

**\*\*Outcome\*\***

The system becomes a real multi-agent orchestration pipeline rather than a linear script.

## Phase 5: Add RAG

1. Create `runbooks/` with mock SOP documents.
2. Build lexical retrieval first.
3. Add optional embedding search second.
4. Pass retrieved context to RCA and remediation agents.

**\*\*Outcome\*\***

The system demonstrates context-aware recommendations and operational memory.

## Phase 6: Add delivery surfaces

1. Expose FastAPI `/analyze`.
2. Build Gradio upload dashboard.
3. Add n8n workflow calling FastAPI.
4. Add Slack and JIRA preview or real integration.

**\*\*Outcome\*\***

The system is demoable by judges from a hosted URL.

# Recommended Agent-by-Agent Prompt Strategy

## Classifier Agent prompt structure

- summarize evidence
- infer category
- infer severity
- justify with citations
- mention uncertainty

## Timeline Agent prompt structure

- compress evidence into ordered milestones
- prefer causal transitions over repeated noise
- cite timestamps and source lines

## **RCA Agent prompt structure**

- separate symptom from cause
- list strongest cause hypothesis
- list alternatives
- identify missing evidence

## **Remediation Agent prompt structure**

- propose safest first actions
- add validation checks
- add rollback notes
- prioritize low-risk reversible actions

## **Critic Agent prompt structure**

- reject unsupported claims
- reject actions with no evidence basis
- flag severity overstatement
- flag remediation without validation checks

## **Notification Agent prompt structure**

- make message concise
- preserve urgency
- remove unnecessary jargon
- include next action and owner cue

## **Ticket Agent prompt structure**

- make issue reproducible for responders
- include evidence and scope
- include remediation checklist and impact summary

## **Cookbook Agent prompt structure**

- generalize from the incident
- extract reusable checklist steps
- keep operational wording

## Best Team Split

For your current team plan, the cleanest split is:

### Engineer 1

- `Classifier Agent`
- `Timeline Agent`

### Engineer 2

- `RCA Agent`
- `Remediation Agent`

### Engineer 3

- `Critic Agent`
- `Notification Agent`
- `Ticket Agent`
- `Cookbook Agent`

If only two people are available for agent work:

### Agent Owner A

- classifier
- timeline
- critic

### Agent Owner B

- RCA
- remediation
- notification
- ticket
- cookbook

This split works because:

- Owner A handles analytical state interpretation
- Owner B handles decision/action outputs

## Testing Strategy

## Unit tests

- parser handles malformed lines
- signal rules detect expected categories
- clusterer groups related evidence
- classifier fallback returns valid category/severity
- ticket routing triggers only for critical incidents

## Agent tests

- every agent returns a valid Pydantic model
- every agent handles empty or weak evidence gracefully
- critic flags unsupported RCA claims
- remediation includes validation checks

## Integration tests

- upload sample logs -> full pipeline output
- no API key -> deterministic fallback still works
- P1 incident -> Slack/JIRA preview generated
- P3 incident -> no JIRA by default

# Risk Analysis

## Biggest risk 1: Too many "agents"

If the team tries to build too many agent types, the project will look impressive on paper but remain unstable.

**\*\*Recommendation\*\***

Keep true LLM agents to the eight listed above.

## Biggest risk 2: Making ingestion agentic

This will waste time and reduce reliability.

**\*\*Recommendation\*\***

Keep parsing, signals, clustering, and retrieval deterministic.

## Biggest risk 3: Real external integrations

Slack and JIRA auth can consume a lot of hackathon time.

**\*\*Recommendation\*\***

Ship preview-first. Add real writes only if the core system is already stable.

## Biggest risk 4: RAG complexity

Full vector DB setup can become a distraction before the deadline.

**\*\*Recommendation\*\***

Ship lexical retrieval first. Mention Weaviate as the target evolution.

## Final Recommendation

For this hackathon, IncidentIQ should present itself as:

> a multi-agent incident intelligence system where deterministic evidence extraction feeds a focused set of specialized reasoning agents, with RAG-backed remediation and cross-tool delivery through Gradio, FastAPI, and n8n.

That story is strong technically and realistic operationally.

The strongest agent lineup for tomorrow is:

1. Classifier
2. Timeline
3. RCA
4. Remediation
5. Critic
6. Notification
7. Ticket
8. Cookbook

Everything else should support those agents, not compete with them.

## Deliverables Suggested For Judges

- hosted Gradio app
- FastAPI backend
- exported n8n workflow
- README with architecture and setup
- final report PDF
- under-5-minute demo video